parameters to improve alignment with specific tasks. However, entirely updating all parameters of an LLM may be impractical due to hardware limitations and potential performance degradation. Therefore, the challenge for researchers lies in identifying which parameters require alteration within the expansive parameter space, or in efficiently updating a subset of these parameters. These two approaches allow LLMs to be tailored to specific tasks or domains, offering flexibility and efficiency in handling specialized applications.

5.1 Adapter-based Fine-tuning

Adapter-based fine-tuning aims to add a small number of extra parameters to the LLM for achieving better performance in specific tasks. Typically the additional parameters are encoded in simple modules to *guide* the language model's adaptation to target domains or tasks. The golden spots for the added modules include: (1) simple with a small number of parameters; (2) extensible to the original language models; (3) flexible with sequential training on each specific domain. Most of the proposed strategies with the above favorable properties are built on adapters [49, 129] under the umbrella of parameter-efficient fine-tuning.

5.1.1 Adapters. Adapters are trainable modules inserted between layers of a pre-trained model [49]. The key property of adapters highlights that the parameters of the original language model keep frozen, thus provide sustainable parameter sharing even with varying domains and tasks. Suppose $f_\Theta(\cdot)$ denotes the function of LLM parametrized with the set of parameters Θ and $g_{\Delta\Theta}(\cdot)$ denotes the function of adapters with parameter $\Delta\Theta$, then $f_\Theta \circ g_{\Delta\Theta}$ represents the fine-tuned language model by adapters. Let X be general input data with task performance metric ϕ, and D be the domain training data with domain-specific task performance ϕ_D (for both ϕ and ϕ_D, a higher value indicates better performance), the goal of adapters is to find $g_{\Delta\Theta}$ such that:

$$\phi(f_\Theta(X)) \approx \phi(f_\Theta \circ g_{\Delta\Theta}(X)) \qquad \phi_D(f_\Theta(D)) \leq \phi_D(f_\Theta \circ g_{\Delta\Theta}(D))$$

Despite most empirical studies on cross-lingual or multi-task learning, some recent works explore unsupervised domain adaptation particularly using adapters. Unsupervised Domain Adaptation (UDA) using adapters has been explored in recent work, aiming to enhance the cross-lingual or multi-task learning capabilities of pre-trained models. The first attempt [183] targeted multi-domain adaptation with a two-step strategy: domain-fusion training with Masked Language Model (MLM) loss on a mixed corpus, followed by task fine-tuning with a task-specific loss on the domain corpus. Subsequently, UDApter was introduced, which also adopted the two-step training and fine-tuning approach, but segregated this into two adapter modules: a domain adapter and a task adapter. The domain adapter first learned domain-invariant representations, which were then concatenated with the task adapter whose parameters were frozen [92]. This was achieved using the architecture defined in AdapterFusion [114]. AdapterSoup further improved adaptation efficiency by adopting a weight-average of domain adapters only during the testing phase [22]. To select domain adapters, three strategies were explored: exhaustive combination, text clustering, and semantic similarity.

Though these works focused on domain specialization, they were evaluated on pre-trained language models like GPT-2 [22, 92, 183], indicating potential applicability to larger language models. To address this, LLaMA-adapter was designed for efficient adaptation on Large Language Models with Adapters (LLaMAs) using self-instruct demonstrations. The adapter architecture incorporated a zero-init attention mechanism, and the domain specialization capability was tested on instruction-following and multi-modal reasoning tasks [182].

As the application of adapters expands, several techniques, while not explicitly claimed as effective for domain specialization, have either demonstrated potential by offering favorable performance on downstream tasks or served

as integrated components in existing frameworks for domain specialization. Hence, adapters are usually classified based on their architectures into neural adapters and low-rank adapters. With the objective of facilitating user-friendly implementation, a growing body of work is dedicated to building comprehensive frameworks of different adapters [54, 115]. Certain studies have also shown that adapter integration can yield superior performance across a variety of downstream tasks.

Neural adapters. We call adapters with neural network architectures neural adapters. In their original design, [49] uses a composition of down-projection, GeLU non-linearity [47] and up-projection with the feed-forward layers as the backbone. Later [6] simplifies the architecture to a single hidden-layer feed-forward network and demonstrates the effectiveness on domain adaptation. The adapter modules are inserted after the multi-head attention and feed-forward layers in the transformer. These adapters have been named as bottleneck adapters or serial adapters. We use the latter throughout this paper when referring to [49].

The development of neural adapters naturally takes inspiration from neural network architecture design, such as ResNet, autoencoder, attention mechanism, etc. The adapters used in [114] have an additional residual connection. Soon after, [116] proposes MAD-X framework with invertible adapters, which are inserted adjacent to input and inverted to be fed into the output embeddings. At the high-level, invertible adapters can be considered a mimic of autoencoders. Tiny-attention adapter [185] explores the effectiveness of adapters using attention with tiny per-head dimensionality. Till now, most proposed architectures apply fully-connected layers for down-projection and up-projection in adapters. However, Compacters [66] considers parameterized hypercomplex multiplication layers [179] as an alternative, which has a similar form as a fully-connected layer, but learns a sum of Kronecker products. The main advantage is parameter efficiency. Another way of achieving this is inspired by network pruning, as proposed by SparseAdapter[46] to further reduce the training parameters by pruning at initialization. Note that SparseAdapter is a generic technique applicable to neural adapters. Congregating adapters via insertion can be considered as adaptation *inside* the language models, an alternative is adaptation *outside* the language models. K-adapters [158] proposes to train multiple adapters individually on various knowledge domains, then inject the learned knowledge with language models by concatenation. Recently, Sung et al. [147] raises a concern on the high training memory required because the backpropagation flows throug the language model with inserted adapters in entirety. They further propose ladder side-tuning, which only adds small modules on the side of the language model connected to the language model backbone via shortcuts. Both techniques use MLP for demonstration, but keep flexible with different adapter architectures.

Low-rank adapters. Low-rank adaptation (LoRA) [52] is inspired by the observation that large language models reside on an intrinsic subspace [**?**], where model parameters are efficiently updated. Therefore, learning in this subspace significantly reduces the amount of parameters. LoRA modules implant learnable SVD blocks as the subspace with a low matrix rank $r \ll d$, where d is the dimension of input data. The matrices are added in parallel to the pre-trained weights, thus keeping them frozen during the fine-tuning. Notably, LoRA shows superiority in further reducing the number of trained parameters and introducing no latency during inference.

A follow-up work on this line is DyLora [150], which addresses two issues of LoRA using dynamic search: fixed block size and exhaustive search on the optimal rank. Recently, another concern of LoRA was raised that low-rank modules have limited representation power, and further resolved by the Kronecker adapter (KronA) [34]. The essence is to substitute the SVD modules with a Kronecker product module with two matrices of smaller sizes. Despite not many follow-ups on the low-rank adapters, LoRA modules are included in various integrated adaptation frameworks [45, 54, 93, 162] as an important building block. More details on these frameworks follow below.

Integrated adapter framework. With the flourishing results on effective adapters as introduced above, it is a natural extension to incorporate several adapters of various families to boost their performance. AdapterFusion [114] employs a straightforward idea: train multiple adapters on different tasks and combine the learned embeddings from each adapter with a fusion layer. UniPELT [93] proposes to activate different combinations of methods that best suit the current data or task setup via a gating mechanism. The sub-modules included serial adapter [49], LoRA [52], Prefix-tuning [79] and Bitfit [178]. Orthogonal to UniPELT, AdaMix [162] stacks multiple adapters of the same type, but avoids more computational cost by training the activation with stochastic routing. AdaMix can be regarded as a general technique that applies to any adapter, despite their implementation on only serial adapters and LoRA.

The idea of learning a routing function on an inventory of adapters further inspires follow-up works. In the context of multi-task learning, Polytropon [117] jointly learns an inventory of adapters and a routing function to re-combine the fine-tuned adapters of various sizes shared among different tasks. Variants of this scheme are further studied by [12], including the replacement of the routing function with weights averaging, or multi-head routing function to achieve better expressivity. On the implementation-oriented aspect, AdapterHub [115] is the most comprehensive and easy-to-use library integrating all mainstream adapters. The only downside, however, is the absence of support of large language models. Recently, LLM-adapters [54] introduces a framework including open-access large language models such as LLaMA, OPT, GPT-J, etc. It subsumes four adapters as basic components (Serial adapter [49], MAD-X [116], Parallel adapter [176] and LoRA [52]) and remains extensible to new modules. The study of domain specialization further explores mathematical reasoning.

5.1.2 Open Challenges. The adapter's wide applications stem from its modular compatibility with language models, flexible design for integration, and efficient domain-specific data fine-tuning, advancing the adapter-based fine-tuning paradigm. However, these methods have drawbacks. Firstly, the performance of inserted modules can be sensitive to architectural design and size across different tasks and domains, risking insufficient representational power or overfitting on limited data. Secondly, additional modules enlarge the model size, imposing new resource demands and possibly extending inference time. Lastly, as Sung et al. note, the training memory needed by adapter-based methods remains substantial as backpropagation involves the entire model even when previous parameters are frozen [147]. Given these discussions, we outline the open challenges in applying adapters to LLMs for domain specialization:

(1) *Stability and universality:* The performance of adapters can be subject to various architecture or hyper-parameters applied even on pre-trained language models (PTLMs), thus imposes a question mark on the stability and universality. This concern further extends to LLMs. A deeper understanding on how different adapters match with different task settings would be a significant boost to broader applications of adapters.
(2) *Computational resources:* Adapters have shown remarkable results with a million-size of parameters on PTLMs. However, it remains unproven if they are enough for LLMs. If more adapter modules (more parameters) are required, then the issue of computational cost can be raised again. Another ideal spot on this issue is to reduce the training memory with novel architecture design or fine-tuning strategy.

5.2 Task-oriented Fine-tuning

Despite these incredible capabilities of LLMs trained on large text corpus, fundamentally improving the model performances beyond few-shot examples and auxiliary adapters still requires updating the inner parameters of LLMs on an extensive amount of high-quality domain-specific datasets. However, fine-tuning an LLM on any (domain) specific tasks poses two challenges: 1) updating LLM's global knowledge may destroy the in-context learning ability due to